

BÚSQUEDA LOCAL Y METAHEURÍSTICAS

Ascenso de colinas y temple simulado

Dos estrategias de búsqueda local para problemas de optimización

Propósito de aprendizaje

Al finalizar la sesión

El estudiante podrá formular un problema como búsqueda local, implementar ascenso de colinas y temple simulado, y justificar cuál método conviene según la forma del espacio de búsqueda.

Conceptos

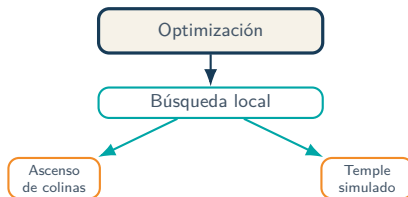
Vecindad, función objetivo, óptimos locales, temperatura y probabilidad de aceptación.

Desempeño esperado

Interpretar trazas de ejecución y analizar el balance entre intensificación y exploración.

Ubicación dentro de los algoritmos inspirados en la naturaleza

- ▶ **Ascenso de colinas:** búsqueda local inspirada en la idea de subir por un paisaje de aptitud.
- ▶ **Temple simulado:** metaheurística inspirada en el recocido de metales y la mecánica estadística.
- ▶ Ambos exploran una vecindad y conservan poca información del recorrido.



El problema de optimización

Sea S el conjunto de soluciones factibles y $f : S \rightarrow \mathbb{R}$ la función objetivo.

$$s^* = \arg \max_{s \in S} f(s)$$

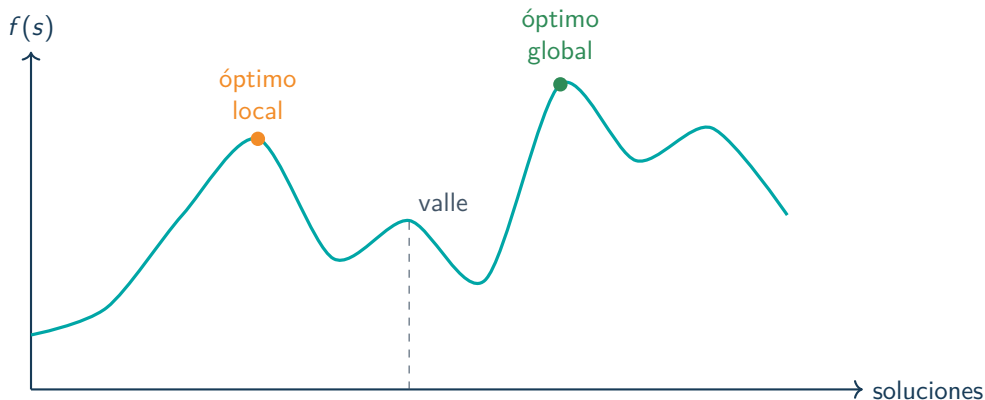
Elementos del modelo

- ▶ Estado o solución candidata s .
- ▶ Vecindad $N(s)$ generada por cambios pequeños.
- ▶ Calidad o aptitud $f(s)$.

Ejemplos

- ▶ Orden de ciudades en una ruta.
- ▶ Horario de clases sin conflictos.
- ▶ Selección de variables de un modelo.

Paisaje de búsqueda



Una decisión basada solo en mejoras inmediatas puede detenerse antes de alcanzar la mejor solución del espacio.

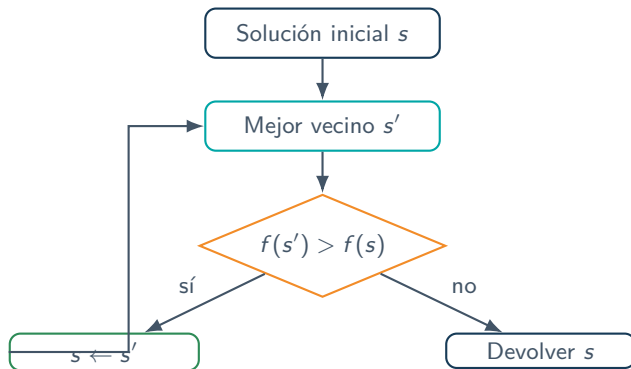
Ascenso de colinas

Idea central

Partir de una solución y reemplazarla por un vecino mejor. El proceso termina cuando ningún vecino ofrece mejora.

1. Crear o recibir una solución inicial.
2. Generar soluciones vecinas.
3. Seleccionar un vecino con mayor valor objetivo.
4. Detenerse si no existe mejora.

Flujo del ascenso de colinas



Pseudocódigo: ascenso más pronunciado

```
def ascenso_colinas(solucion_inicial):  
    actual = solucion_inicial  
  
    while True:  
        vecinos = generar_vecinos(actual)  
        mejor = max(vecinos, key=funcion_objetivo)  
  
        if funcion_objetivo(mejor) <= funcion_objetivo(actual):  
            return actual  
  
    actual = mejor
```

Si cada estado tiene b vecinos y se ejecutan k iteraciones, el costo aproximado es $O(kb)$ evaluaciones de la función objetivo.

Variantes del ascenso de colinas

Variante	Regla de movimiento
Simple	Acepta el primer vecino que mejora la solución actual.
Más pronunciado	Evalúa la vecindad y elige la mayor mejora.
Estocástico	Elige de manera aleatoria entre los vecinos que mejoran.
Con reinicios	Ejecuta el algoritmo desde varias soluciones iniciales.

Los reinicios aleatorios aumentan la cobertura del espacio sin cambiar la regla local de movimiento.

Limitaciones del ascenso de colinas

Óptimo local

Todos los vecinos son peores, aunque exista una solución global superior.

Meseta

Varios vecinos tienen la misma calidad y no indican una dirección clara.

Cresta

La mejora exige una secuencia de movimientos que el operador local no representa bien.

Consecuencia

El resultado depende de la solución inicial y de cómo se define la vecindad.

Ejemplo: maximización en una dimensión

Considere $f(x) = x \sin(3x) + 2$, con $0 \leq x \leq 6$, y vecinos $x \pm 0.1$.

- ▶ Estado inicial: $x_0 = 0.8$.
- ▶ El algoritmo compara los dos vecinos.
- ▶ Cada paso conserva la mejor mejora.
- ▶ La ejecución termina en un máximo cercano, que puede no ser global.

Pregunta de análisis

¿Cambiaría el resultado si la búsqueda comenzara en $x_0 = 4.5$?

Idea central

El algoritmo permite algunos movimientos que empeoran la solución. Esta tolerancia disminuye a medida que baja la temperatura.

- ▶ A temperatura alta, la búsqueda explora regiones diferentes.
- ▶ A temperatura baja, el comportamiento se aproxima al ascenso de colinas.
- ▶ El algoritmo conserva la mejor solución encontrada, aunque el estado actual empeore.

Analogía con el recocido de metales

Proceso físico

- ▶ El material se calienta.
- ▶ Los átomos alcanzan alta movilidad.
- ▶ El enfriamiento gradual favorece una estructura estable.

Proceso computacional

- ▶ La temperatura controla la aleatoriedad.
- ▶ Los movimientos peores permiten salir de óptimos locales.
- ▶ El enfriamiento concentra la búsqueda en soluciones de alta calidad.

El nombre proviene del trabajo de Kirkpatrick, Gelatt y Vecchi publicado en 1983.

Regla de aceptación

Para un problema de minimización, sea

$$\Delta E = f(s') - f(s).$$

- ▶ Si $\Delta E \leq 0$, se acepta el vecino.
- ▶ Si $\Delta E > 0$, se acepta con probabilidad $P(\text{aceptar}) = e^{-\Delta E/T}$.

Interpretación

Una temperatura alta o un deterioro pequeño producen mayor probabilidad de aceptación.

Probabilidad de aceptar una solución peor

ΔE	T	$e^{-\Delta E/T}$	Lectura
1	10	0.905	Muy probable
5	10	0.607	Posible
5	1	0.007	Poco probable
10	1	0.00005	Casi imposible

La misma pérdida puede aceptarse al inicio y rechazarse cerca del final de la ejecución.

Programa de enfriamiento

La temperatura cambia según una función $T(k)$, donde k representa la iteración.

Enfriamiento geométrico

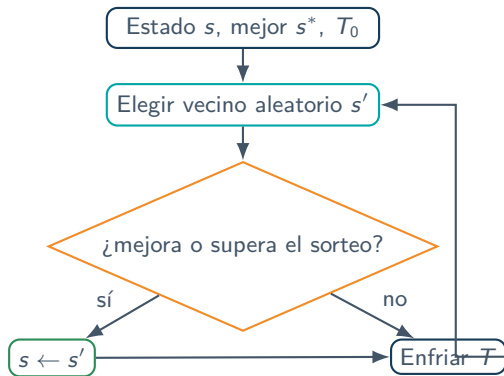
$$T_{k+1} = \alpha T_k, \quad 0 < \alpha < 1$$

Es sencillo y frecuente. Valores de α cercanos a 1 producen un enfriamiento lento.

Decisiones de diseño

- ▶ Temperatura inicial T_0 .
- ▶ Factor de enfriamiento α .
- ▶ Iteraciones por temperatura.
- ▶ Criterio de terminación.

Flujo del temple simulado



Pseudocódigo: temple simulado

```
def temple_simulado(inicial, T, alpha, T_min):  
    actual = mejor = inicial  
    while T > T_min:  
        candidato = vecino_aleatorio(actual)  
        delta = costo(candidato) - costo(actual)  
  
        if delta <= 0 or random() < exp(-delta / T):  
            actual = candidato  
            if costo(actual) < costo(mejor):  
                mejor = actual  
  
        T = alpha * T  
    return mejor
```

Efecto de los parámetros

Decisión	Valor extremo	Efecto probable
T_0 muy baja	Pocas aceptaciones adversas	Convergencia prematura
T_0 muy alta	Movimiento casi aleatorio	Inicio poco selectivo
α pequeño	Enfriamiento rápido	Menor tiempo, menor exploración
α cercano a 1	Enfriamiento lento	Mayor costo, mejor cobertura
Vecindad limitada	Cambios muy pequeños	Dificultad para cambiar de región

Comparación de los dos algoritmos

Criterio	Ascenso de colinas	Temple simulado
Movimiento peor	Se rechaza	Puede aceptarse
Selección del vecino	Mejor o primer vecino superior	Vecino usualmente aleatorio
Parámetros	Pocos	Temperatura y enfriamiento
Costo por iteración	Puede evaluar muchos vecinos	Suele evaluar un vecino
Óptimos locales	Alta sensibilidad	Puede escapar de ellos
Reproducibilidad	Depende del inicio	Depende también del azar

Intensificación y exploración

Intensificación

Buscar con detalle alrededor de soluciones prometedoras. El ascenso de colinas prioriza este comportamiento.

Exploración

Visitar regiones diferentes del espacio. El temple simulado explora más cuando la temperatura es alta.

Una metaheurística eficaz administra este balance durante la ejecución.

Problemas combinatorios

- ▶ Rutas y secuencias de visita.
- ▶ Asignación de horarios y recursos.
- ▶ Distribución de tareas.

Modelos de IA

- ▶ Selección de características.
- ▶ Ajuste de hiperparámetros discretos.
- ▶ Diseño de reglas o estructuras.

La función objetivo debe representar la calidad real y penalizar las restricciones incumplidas.

Reto

Maximizar $f(x) = x \sin(3x) + 2$ en $0 \leq x \leq 6$ con pasos de 0.1.

1. Ejecute ascenso de colinas desde cinco puntos iniciales.
2. Ejecute temple simulado con $T_0 = 10$, $\alpha = 0.97$ y $T_{\min} = 0.01$.
3. Repita cada configuración 30 veces.
4. Compare mejor valor, promedio, desviación estándar y evaluaciones.
5. Explique qué evidencia muestra exploración o convergencia prematura.

Preguntas para la discusión

1. ¿Cuándo resulta suficiente el ascenso de colinas con reinicios?
2. ¿Qué ocurre si el temple simulado se enfría demasiado rápido?
3. ¿Cómo afecta una vecindad grande al costo y a la calidad?
4. ¿Una mejor solución en una sola ejecución demuestra que un algoritmo es superior?

Criterio experimental

La comparación requiere múltiples ejecuciones, el mismo presupuesto computacional y métricas definidas antes del experimento.

- ▶ El ascenso de colinas mejora con rapidez, pero puede detenerse en óptimos locales.
- ▶ Los reinicios reducen la dependencia de una única solución inicial.
- ▶ El temple simulado acepta deterioros controlados para atravesar valles del paisaje.
- ▶ El programa de enfriamiento determina la transición entre exploración e intensificación.
- ▶ Ningún método garantiza por sí solo la mejor respuesta en tiempo finito para cualquier problema.

Referencias

- ▶ Aarts, E., Korst, J., y Michiels, W. (2005). Simulated annealing. En *Search Methodologies*. Springer.
- ▶ Kirkpatrick, S., Gelatt, C. D., y Vecchi, M. P. (1983). Optimization by simulated annealing. *Science*, 220(4598), 671–680.
- ▶ Luke, S. (2013). *Essentials of Metaheuristics* (2.a ed.). Lulu.
- ▶ Russell, S., y Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4.a ed.). Pearson.

¿Preguntas?